

Compact block relay reconstruction specification.

Models the draft’s “Compact Block Relay” machinery between a sender S (who has the new block) and a requester R (who holds some of its transactions): compact block announcements carrying a per-announcement nonce, short-ID matching against R ’s *mempool*, *get-tx SHORTID requests for the missing transactions*, and the reconstruction outcome.

The stateful subtlety is the nonce : short transaction IDs are relative to one compact block’s nonce, and the responding node interprets *SHORTID* references “using the nonce of the compact block it most recently sent to the requesting peer for the identified block” (“Requesting Missing Transactions”). A sender may legitimately announce the same block more than once with a fresh nonce — for example when re-announcing its tip on a replacement announcement stream (see Finding 2 in [../documents/v2-modeling.md](#)). *SHORTID* requests computed from an earlier announcement then arrive under a newer nonce: each reference matches no transaction (not-found) or, by 48-bit collision, a wrong one. The model resolves that choice adversarially.

Announcements are best-effort: under backpressure the sender may drop one rather than queue it (“Announcement Streams”), and a reset replacement stream loses records in flight. The model lets a re-announcement be lost while the requester is mid-attempt — after which every reference the requester can ever send is stale, and the compact path alone cannot converge.

The draft’s escape hatch is reconstruction failure handling: the node “SHOULD fall back to requesting the full block via *get-blocks*, and MUST NOT assign a misbehavior penalty solely because reconstruction failed”. Two switches probe it:

Fallback = FALSE → reconstruction failure retries the compact path instead of falling back. Liveness (*EventuallyHasBlock*) is violated: announce budgets end and the requester waits forever. The SHOULD is load-bearing — no other rule guarantees the block arrives.

PenalizeReconFail = TRUE → reconstruction failure scores 20 points, against the MUST NOT. *NoHonestPenalty* is violated: honest nonce churn frames the sender.

With *Fallback* = TRUE and no penalties, every property holds under maximal nonce churn, and *WrongTxNeverAccepted* states the draft’s own claim that short-ID matching cannot weaken consensus: a wrongly matched transaction fails the merkle check and never enters an accepted block.

See [../documents/v2-modeling.md](#) for the full write-up.

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT NTx	transactions in the block: $1 \dots NTx$
CONSTANT $MaxAnnounce$	compact block announcements available to S
CONSTANT $Fallback$	see module comment
CONSTANT $PenalizeReconFail$	see module comment

$Txs \triangleq 1 \dots NTx$

VARIABLES

$mempool$,	R: subset of Txs held before the block arrives
has_block ,	R: TRUE once the block is accepted (fully reconstructed or fetched)
$score$,	R’s misbehavior score for S (MUST stay 0)

<i>ann_q</i> ,	announcement stream toward <i>R</i> : sequence of nonces (in order)
<i>last_sent</i> ,	<i>S</i> : nonce of the compact block most recently sent to <i>R</i> ($0 = \text{none}$)
<i>announced</i> ,	<i>S</i> : announcements used
<i>via</i> ,	how <i>R</i> got the block: “none” “match” “recon” “full” (ghost)
<i>req</i> ,	<i>SHORTID</i> get-tx toward <i>S</i> : [active, nonce, missing]
<i>resp</i> ,	get-tx response toward <i>R</i> : [active, <i>ok</i> : txs served correctly, bad : a not-found or wrong tx occurred]
<i>fb</i>	fallback get-blocks state: “none” “requested” “served”

$\text{vars} \triangleq \langle \text{mempool}, \text{has_block}, \text{score}, \text{ann_q}, \text{last_sent}, \text{announced}, \text{via}, \text{req}, \text{resp}, \text{fb} \rangle$

$\text{NoReq} \triangleq [\text{active} \mapsto \text{FALSE}, \text{nonce} \mapsto 0, \text{missing} \mapsto \{\}]$

$\text{NoResp} \triangleq [\text{active} \mapsto \text{FALSE}, \text{ok} \mapsto \{\}, \text{bad} \mapsto \text{FALSE}]$

$\text{Init} \triangleq$

$\wedge \text{mempool} \in \text{SUBSET } \text{TxS}$
 $\wedge \text{has_block} = \text{FALSE}$
 $\wedge \text{score} = 0$
 $\wedge \text{ann_q} = \langle \rangle$
 $\wedge \text{via} = \text{“none”}$
 $\wedge \text{last_sent} = 0$
 $\wedge \text{announced} = 0$
 $\wedge \text{req} = \text{NoReq}$
 $\wedge \text{resp} = \text{NoResp}$
 $\wedge \text{fb} = \text{“none”}$

S announces the block as a compact block with a fresh nonce — the first announcement, or a legitimate re-announcement (replacement stream, *BIP* 152 high-bandwidth relay). Nonces are the announcement number.

$\text{Announce} \triangleq$

$\wedge \text{announced} < \text{MaxAnnounce}$
 $\wedge \text{announced}' = \text{announced} + 1$
 $\wedge \text{ann_q}' = \text{Append}(\text{ann_q}, \text{announced} + 1)$
 $\wedge \text{last_sent}' = \text{announced} + 1$
 $\wedge \text{UNCHANGED } \langle \text{mempool}, \text{has_block}, \text{score}, \text{via}, \text{req}, \text{resp}, \text{fb} \rangle$

R processes the next compact block announcement: transactions it holds match by short *ID*; the missing ones are requested by *SHORTID* reference, remembering which nonce they were computed under. If nothing is missing the block is reconstructed immediately. Announcements for a block already held are ignored.

$\text{RecvCompactBlock} \triangleq$

$\wedge \text{ann_q} \neq \langle \rangle$
 $\wedge \neg \text{req.active} \wedge \neg \text{resp.active} \wedge \text{fb} = \text{“none”}$
 $\wedge \text{ann_q}' = \text{Tail}(\text{ann_q})$

$\wedge$ IF has_block
 THEN UNCHANGED $\langle mempool, has_block, score, last_sent, announced, via, req, resp, fb \rangle$
 ELSE IF $Txs \subseteq mempool$
 THEN $\wedge has_block' = \text{TRUE}$
 $\wedge via' = \text{"match"}$
 $\wedge$ UNCHANGED $\langle mempool, score, last_sent, announced, req, resp, fb \rangle$
 ELSE $\wedge req' = [active \mapsto \text{TRUE}, nonce \mapsto \text{Head}(ann_q), missing \mapsto Txs \setminus mempool]$
 $\wedge$ UNCHANGED $\langle mempool, has_block, score, last_sent, announced, via, resp, fb \rangle$

A queued re-announcement is lost while the requester is busy with an attempt: announcements are best-effort under backpressure, and a reset replacement stream drops records in flight. Adversarial, never forced.

$DropAnnouncement \triangleq$
 $\wedge ann_q \neq \langle \rangle$
 $\wedge req.active \vee resp.active$
 $\wedge ann_q' = Tail(ann_q)$
 $\wedge$ UNCHANGED $\langle mempool, has_block, score, last_sent, announced, via, req, resp, fb \rangle$

S serves the *SHORTID* references, interpreting them under the nonce of the compact block it most recently sent. Fresh references resolve exactly; stale ones (an announcement has happened since) match no transaction or, by collision, a wrong one — resolved adversarially per reference.

$ServeGetTx \triangleq$
 $\wedge req.active$
 $\wedge \exists badset \in \text{SUBSET } req.missing :$
 $\wedge (req.nonce = last_sent) \Rightarrow badset = \{\}$
 $\wedge (req.nonce \neq last_sent) \Rightarrow badset \neq \{\}$
 $\wedge resp' = [active \mapsto \text{TRUE}, ok \mapsto req.missing \setminus badset, bad \mapsto badset \neq \{\}]$
 $\wedge req' = NoReq$
 $\wedge$ UNCHANGED $\langle mempool, has_block, score, ann_q, last_sent, announced, via, fb \rangle$

R completes reconstruction from a clean response: the block passes its merkle check and is accepted.

$RecvTxComplete \triangleq$
 $\wedge resp.active$
 $\wedge \neg resp.bad$
 $\wedge mempool' = mempool \cup resp.ok$
 $\wedge has_block' = \text{TRUE}$
 $\wedge via' = \text{"recon"}$
 $\wedge resp' = NoResp$
 $\wedge$ UNCHANGED $\langle score, ann_q, last_sent, announced, req, fb \rangle$

A not-found result or a wrong transaction makes reconstruction fail (a wrong transaction fails the merkle check). What happens next is the switch: fall back to get-blocks as the draft says SHOULD, or retry the compact path with the next announcement; and, forbidden by the draft's

MUST NOT, a penalty may be scored.

$$\begin{aligned}
RecvTxFailed &\triangleq \\
&\wedge \text{resp.active} \\
&\wedge \text{resp.bad} \\
&\wedge \text{mempool}' = \text{mempool} \cup \text{resp.ok} \\
&\wedge \text{resp}' = \text{NoResp} \\
&\wedge \text{fb}' = \text{IF } \text{Fallback} \text{ THEN "requested" ELSE "none"} \\
&\wedge \text{score}' = \text{IF } \text{PenalizeReconFail} \text{ THEN } \text{score} + 20 \text{ ELSE } \text{score} \\
&\wedge \text{UNCHANGED } \langle \text{has_block}, \text{ann_q}, \text{last_sent}, \text{announced}, \text{via}, \text{req} \rangle
\end{aligned}$$

The fallback get-blocks round trip: S always holds the full block.

$$\begin{aligned}
ServeFallback &\triangleq \\
&\wedge \text{fb} = \text{"requested"} \\
&\wedge \text{fb}' = \text{"served"} \\
&\wedge \text{UNCHANGED } \langle \text{mempool}, \text{has_block}, \text{score}, \text{ann_q}, \text{last_sent}, \text{announced}, \text{via}, \text{req}, \text{resp} \rangle
\end{aligned}$$

$$\begin{aligned}
RecvFallback &\triangleq \\
&\wedge \text{fb} = \text{"served"} \\
&\wedge \text{has_block}' = \text{TRUE} \\
&\wedge \text{via}' = \text{"full"} \\
&\wedge \text{fb}' = \text{"none"} \\
&\wedge \text{UNCHANGED } \langle \text{mempool}, \text{score}, \text{ann_q}, \text{last_sent}, \text{announced}, \text{req}, \text{resp} \rangle
\end{aligned}$$

$$\begin{aligned}
Next &\triangleq \\
&\vee \text{Announce} \\
&\vee \text{RecvCompactBlock} \\
&\vee \text{DropAnnouncement} \\
&\vee \text{ServeGetTx} \\
&\vee \text{RecvTxComplete} \\
&\vee \text{RecvTxFailed} \\
&\vee \text{ServeFallback} \\
&\vee \text{RecvFallback}
\end{aligned}$$

Everything a peer must eventually do is weakly fair; *Announce* is fair too, so the block is announced at all, and its budget bounds the churn. *DropAnnouncement* carries no fairness: it is adversarial, never forced — *WF* on the other actions cannot force it either, since *RecvCompactBlock* is enabled whenever it is.

$$\begin{aligned}
Spec &\triangleq \\
&\text{Init} \\
&\wedge \Box [Next]_{vars} \\
&\wedge \text{WF}_{vars}(\text{Announce}) \\
&\wedge \text{WF}_{vars}(\text{RecvCompactBlock}) \\
&\wedge \text{WF}_{vars}(\text{ServeGetTx})
\end{aligned}$$

$$\begin{aligned}
& \wedge \text{WF}_{vars}(\text{RecvTxsComplete}) \\
& \wedge \text{WF}_{vars}(\text{RecvTxsFailed}) \\
& \wedge \text{WF}_{vars}(\text{ServeFallback}) \\
& \wedge \text{WF}_{vars}(\text{RecvFallback})
\end{aligned}$$

Liveness: R eventually holds the block.
 $\text{EventuallyHasBlock} \triangleq \Diamond \text{has_block}$

Safety invariants.

$$\begin{aligned}
\text{TypeOK} & \triangleq \\
& \wedge \text{mempool} \subseteq \text{Txs} \\
& \wedge \text{has_block} \in \text{BOOLEAN} \\
& \wedge \text{score} \in \text{Nat} \\
& \wedge \text{last_sent} \in 0 \dots \text{MaxAnnounce} \\
& \wedge \text{announced} \in 0 \dots \text{MaxAnnounce} \\
& \wedge \text{fb} \in \{\text{"none"}, \text{"requested"}, \text{"served"}\} \\
& \wedge \text{via} \in \{\text{"none"}, \text{"match"}, \text{"recon"}, \text{"full"}\}
\end{aligned}$$

The draft's MUST NOT: reconstruction failure carries no penalty.
 $\text{NoHonestPenalty} \triangleq \text{score} = 0$

The draft's consensus claim: short-ID matching cannot weaken consensus.
A block accepted through reconstruction was assembled from the full,
correct transaction set (bad responses never reach the *mempool* and fail
the merkle check instead); anything else arrived as a full block.

$$\begin{aligned}
\text{WrongTxNeverAccepted} & \triangleq \\
& \text{has_block} \wedge \text{via} \in \{\text{"match"}, \text{"recon"}\} \Rightarrow \text{Txs} \subseteq \text{mempool}
\end{aligned}$$
